Cowside Udder Health Data Collection App

Product Requirements Document & Implementation Plan | Android, offline-first, Supabase backend | v0.1 draft

1. Purpose and goals

Replace the A4 paper Cowside Visit Log with an Android app that captures the same data per farm visit, cow and quarter, works with no connectivity, and syncs to Supabase (Postgres) when a connection is available. The app must map 1:1 to the existing tables: FARM_VISIT, COW_VISIT, QUARTER_VISIT, SAMPLE_LAB and TREATMENT.

- **Goals:** eliminate transcription, enforce valid codes at entry, cut per-cow entry time versus paper, zero data loss on device failure or poor network.
- **Non-goals (v1):** lab result entry, herd management, analytics dashboards, iOS, farmer-facing features.
- **Success metrics:** <3 min per cow record; <2% records failing validation at sync; 100% of submitted visits present in Supabase within 24 h of connectivity; no lost visits in pilot.

2. Users and usage context

- **Observers / field technicians** (primary): record at the cow, often in wet, bright, low-signal conditions, sometimes gloved. Need large tap targets, high contrast, one-hand use.
- **Study admin** (secondary): manages farms, observer accounts and code lists; exports data via Supabase.
- Assumed devices: mid-range Android 9+ phones or small tablets; shared devices possible.

3. Functional requirements

Req	Description
FR1 Auth	Email/password sign-in via Supabase Auth; session cached so the app opens offline. Observer ID derived from the account.
FR2 Farm and cow registry	Farms and cows downloaded to the device for offline pick-lists. Add a new cow in-app (Cow ID, local tag, parity, calving date). Look up by Cow ID or tag.
FR3 Visit header	Create visit (Section A): state/LGA, farm, date, arrival/milking time, milking method, cows present vs expected, floor wetness, bedding, fly abundance, feed/routine changes. Visit ID auto-generated; date defaults to today.
FR4 Cow record	Per cow: tag, parity, calving date with auto-computed DIM, yield and unit, BCS, udder hygiene, rectal temp, mastitis/Rx in past 14 d. Prefill stable fields from the last visit.
FR5 Quarter grid	Four cards (LF, RF, LR, RR): functional/blind, clinical mastitis, grade, milk signs, udder signs, CMT score, lab sample ID, treatment given, notes/missing reason. Blind quarter disables dependent fields.
FR6 Sample IDs	Manual entry or barcode/QR scan; unique per study; duplicate check on device and server. Creates a SAMPLE_LAB row.
FR7 Treatment log	Add treatment entries (cow, quarter(s), grade, sample-before-Tx, product, active ingredient, route/dose, start date, milk withhold until). Withhold date suggested from start date plus product withdrawal period (editable).
FR8 Reference keys	In-app CMT, mastitis grade and hygiene keys shown as bottom sheets on the relevant field.
FR9 Review and submit	Visit summary with validation flags; submitting locks the visit. Post-submit edits by admin only, with audit trail.
FR10 Offline and sync	All reads/writes hit local SQLite. Background sync when online, with visible per-visit status (draft, pending, synced, error) and manual retry.
FR11 Export	Admin CSV export per table via Supabase views; column names match the paper-sheet fields.

4. Validation and derived fields

- Ranges: BCS 1-5 (0.25 steps), hygiene 1-4, grade 1-3, CMT in {N, T, 1, 2, 3}, floor wetness / fly abundance 0-3, rectal temp plausible band (flag outside 36-42 C).
- Grade required if clinical mastitis = yes, and blank if no. Grade 3 prompts a check that temp > 39.5 C or systemic signs are noted; soft warning, not a block.
- Blind quarter: CMT, sample ID, clinical mastitis and treatment fields disabled; notes/missing reason required.
- Any empty required or skipped field needs a missing-reason code (not tested, cow absent, refused, other) so blanks are never ambiguous.
- DIM = visit date minus calving date. Cow count present cannot be less than cows recorded. Treatment start date cannot precede calving or exceed today.

5. Data model (Supabase Postgres)

Reference tables: **farm**, **cow** (farm_id, cow_code, local_tag, parity, calving_date), **observer** (auth user id, name). Visit-level tables below. All tables carry: **id** (UUID generated on device), **created_at**, **updated_at**, **deleted_at** (soft delete), **device_id**, **created_by**.

Table	Key columns
FARM_VISIT	farm_id, visit_code, visit_date, arrival_time, state_lga, milking_method, cows_present, cows_expected, floor_wetness, bedding, fly_abundance, routine_change (bool + text), observers, status
COW_VISIT	farm_visit_id, cow_id, parity, calving_date, dim, yield_value, yield_unit, bcs, udder_hygiene, rectal_temp_c, mastitis_rx_14d
QUARTER_VISIT	cow_visit_id, quarter (LF/RF/LR/RR), functional, clinical_mastitis, grade, milk_signs, udder_signs, cmt_score, tx_given, missing_reason, notes. Unique (cow_visit_id, quarter)
SAMPLE_LAB	quarter_visit_id, sample_id (unique), collected_at, lab_status (defaults to collected; lab fields filled later outside the app)
TREATMENT	cow_visit_id, quarters (array), grade, sample_before_tx, sample_id (nullable), product_name, active_ingredient, route_dose, start_date, milk_withhold_until

Code lists (enums) are Postgres enum types mirrored in the app so entry codes stay identical across device, database and exports. Unique constraints: visit_code, sample_id, (farm_id, visit_date, visit sequence).

6. Architecture

Layer	Decision
Client	Native Android: Kotlin, Jetpack Compose (large-target, high-contrast theme), MVVM with a repository layer.
Local store	Room over SQLite, source of truth for the UI. Optional SQLCipher encryption. Every row has a sync_state (dirty, synced, conflict).
Sync	WorkManager job (network-constrained, exponential backoff). Push: dirty rows in FK order (visit, cow, quarter, sample, treatment) as batched upserts on UUID. Pull: rows changed since last cursor (updated_at) for farms, cows and admin edits. Idempotent, safe to retry.
Conflicts	Visits are single-author so conflicts are rare: last-write-wins by updated_at on drafts; submitted visits are immutable to the client and server-locked. Reference data is server-wins.
Backend	Supabase Postgres, Auth, Row Level Security (observers read/write only own visits and read reference data; admins full access). Server-side check constraints and triggers repeat the validation rules. Audit table for post-submit edits.
Security	TLS in transit, encrypted at rest, no PII beyond farm/observer identifiers, remote sign-out, app lock optional. Confirm data-protection requirements with the study ethics approval.
Ops	Nightly Supabase backups, staging project separate from production, Play internal testing track or direct APK for distribution.

7. Non-functional requirements

- Fully usable offline for at least 7 days and 500 cow records per device; no network call blocks the UI.
- Autosave on every field change; app kill or battery loss loses at most the current field.
- Cold start <3 s; Android 9+ (API 28); usable in direct sunlight and with wet or gloved hands; English UI, strings externalised for later translation.
- Sync of a typical visit (30 cows) completes within 60 s on a weak 3G connection.

8. Implementation plan

Phase	Time	Scope	Output
0. Definition	Wk 1	Freeze data dictionary and code lists against the paper form; agree required-field and missing-reason rules; wireframe cow entry flow with 2 observers.	Signed-off spec
1. Backend	Wk 2-3	Supabase project (dev/prod); schema, enums, constraints; RLS policies; auth users; seed farms and cows; CSV export views.	Schema + RLS tests
2. App core	Wk 3-6	Project setup, Room schema, registry screens, visit header, cow record wizard, quarter grid, treatment log, validation engine, DIM and defaults.	Offline data entry build
3. Sync	Wk 6-8	Sync engine, status UI, retry, conflict handling, submission lock, audit trail; fault tests (airplane mode, mid-sync kill, duplicate sample ID).	End-to-end sync
4. Field pilot	Wk 8-10	2-3 farms, 2 observers, run paper sheet in parallel; compare records; measure time per cow; collect usability issues.	Pilot report, bug list
5. Release	Wk 11-12	Fix pilot findings, barcode scan, encryption, observer training, distribution, backup and support routine.	v1.0 rollout

Estimate assumes one Android developer part-time on backend; scale by team size. Buffer of 2 weeks is not included.

9. Risks and open questions

- **Low connectivity for weeks:** mitigated by offline-first design; storage and battery guidance for observers.
- **Duplicate or mistyped sample IDs:** barcode scan plus uniqueness checks on device and server.
- **Shared devices / lost phones:** per-user login, encrypted local DB, remote sign-out.
- **Scope creep from paper form changes mid-study:** version the form; store form_version on each visit.
- **Open:** number of observers and devices? Need Cow ID barcodes or ear-tag OCR? Lab results to be joined later via sample ID? Required languages? Approved treatment product list for pick-lists?